ELECTRONICS ENGINNERING

MICROPROCCESSORS LABORATORY

2023-2024 FALL

# EXPERIMENT-3

## Group Members

| Name | Fouad | Büşra | Ayşe Beril |
|---|---|---|---|
| Surname | Aladhami | Ülker | Çevik |
| Student ID | 210102002808 | 200102002011 | 2101020002020 |

**1 PROBLEM1 ..... 3**

1.1 FLOWCHART ..... 3
1.2 CODE ..... 3

**2 PROBLEM2 ..... 5**

2.1 FLOWCHART ..... 6
2.2 SCHEMATIC ..... 7
2.3 CODE ..... 7

**3 PROBLEM3 ..... 11**

3.1 FLOWCHART ..... 12
3.2 SCHEMATIC ..... 13
3.3 CODE ..... 13

**4 PROBLEM 4 ..... 20**

4.1 FLOWCHART ..... 22
4.2 CODE ..... 22

**5 PROBLEM 5 ..... 25**

5.1 FLOWCHART ..... 25
5.2 SCHEMATIC ..... 25
5.3 CODE ..... 25

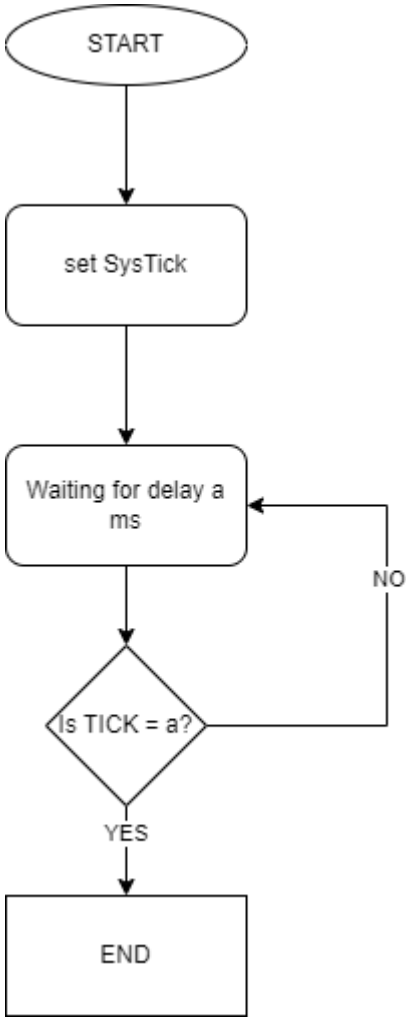
**6 REFERENCES ..... 33**

# 1 PROBLEM1

As a software method we can record SysTick values before and after delay. Hence the difference between these values can be calculated.

An oscilloscope can be used as a hardware method. Calculating the time difference between the toggled signal frequency at the start and end of the delay. -oscilloscope used for measure the frequencies-

## 1.1 FLOWCHART




## 1.2 CODE

```
#include "stm32g0xx.h"
#include "stm32g031xx.h"


//variables
uint64_t tick;
uint8_t seconds;



//functions prototypes
void delay(volatile uint32_t);
void Init_Systick(uint32_t tick);
void Systick_Handler(void);
void increase_tick(void);
uint64_t getTick(void);
void delay_ms(uint64_t msvalue);
void SysTick_Handler(void);

int main(void) {

	//init
	Init_Systick(16000);

	while(1) {
      delay_ms(1000);
      seconds ++;
   }

   return 0;
}
void increase_tick(void){
	tick++;

}
void Init_Systick(uint32_t tick){
	SysTick->LOAD = tick; // Count down from 999 to 0
	SysTick->VAL  = 0;   // Clear current value
	SysTick->CTRL = 0x7; // Enable Systick, exception,and use processor clock
}
uint64_t getTick(void){
	return tick;
}
void delay_ms(uint64_t msvalue){
	uint64_t startTick =getTick() ;
	while ((getTick() - startTick) < msvalue)
```

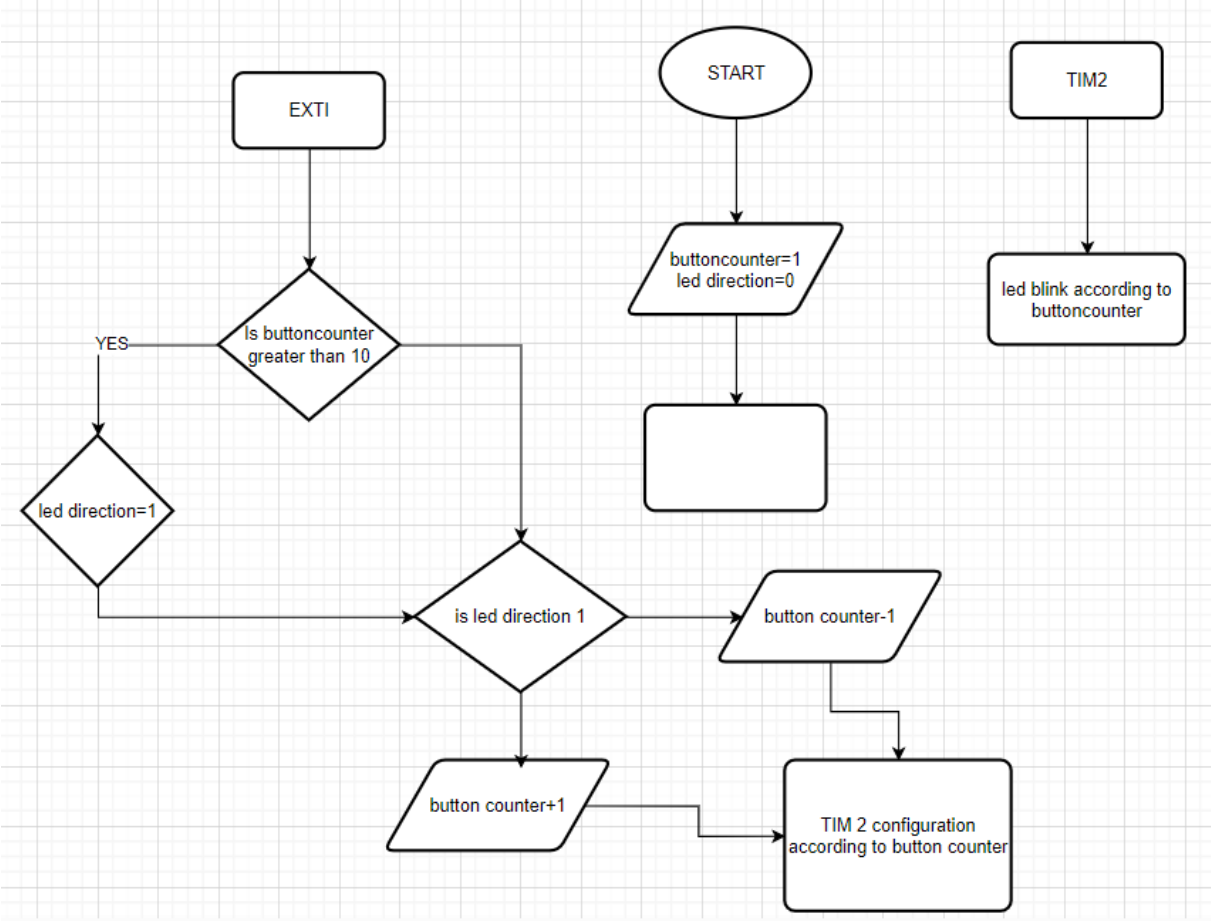
```
          {
          }

}
/**
  * @brief This function handles System tick timer.
  */
void SysTick_Handler(void)
{
          increase_tick();
}
```

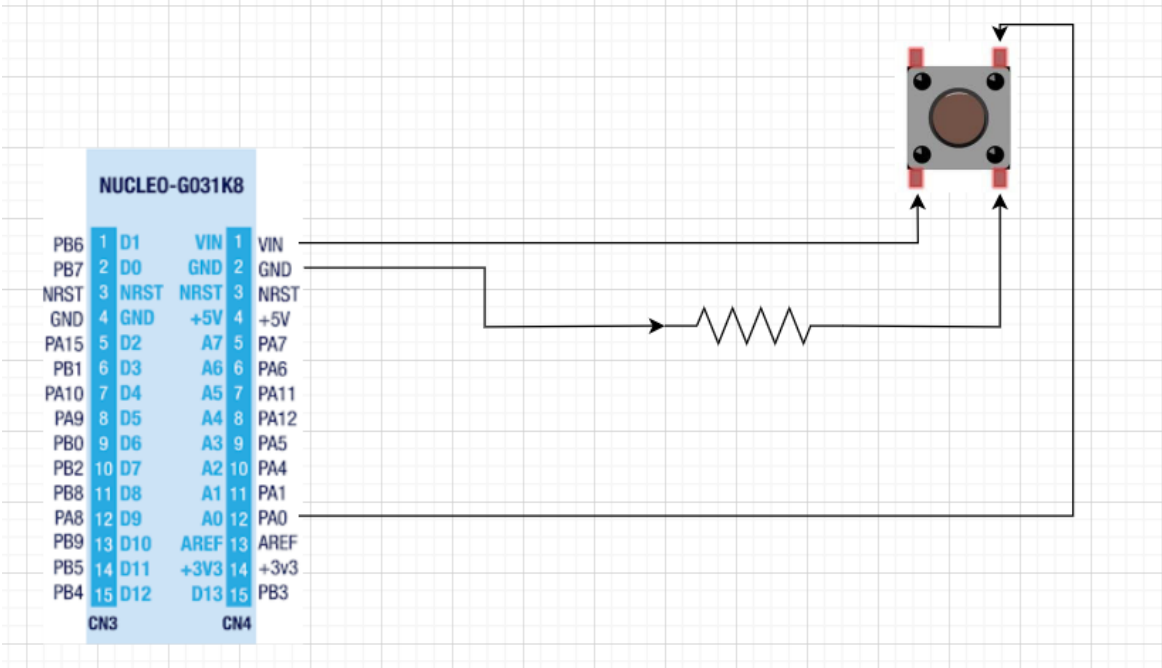
# 2 PROBLEM2

The TIM2 timer interrupt is used to change the state of the LED by tracking the time. The EXTI interrupt is used to increase or decrease the blinking rate of the LED when a button is pressed. As a result, it realises an application that uses timer and external interrupts to control the LED to blink at a specific speed.

## 2.1 FLOWCHART



## 2.2 SCHEMATIC




## 2.3 CODE

```
#include "stm32g0xx.h"
#include "stm32g031xx.h"


//defines
#define TIMERPSC 16000
#define TIMERPERIYOD 1000
//variables
uint64_t tick;
uint64_t seconds;
uint8_t buttonCounter = 1;
uint8_t buttonPreviousState;
uint8_t buttonNewState;
uint8_t ledDirection = 0;

//functions prototypes
void GPIO_RCC_Init(void);
void GPIOC_Init(void);
void GPIOA_Init(void);
```

```
void TIM2_Clock_Init(void);
void TIM2_Interrupt_Config(void);
void TIM2_IRQHandler(void);
void Start_TIM2(void);
void Stop_TIM2(void);

void toggle_led(void);
uint8_t readButton(void);

void EXTI_Init(void);
void EXTI0_1_IRQHandler(void);

int main(void) {

        //init
        GPIO_RCC_Init();
        GPIOA_Init();
        GPIOC_Init();
        TIM2_Clock_Init();
        TIM2_Interrupt_Config();
        EXTI_Init();

        Start_TIM2();

        while(1) {

   }
   return 0;
}

void TIM2_IRQHandler(void) {
   if (TIM2->SR & TIM_SR_UIF) {
      //Code here
          seconds +=  buttonCounter;
          toggle_led();

      // Clear the interrupt flag
      TIM2->SR &= ~TIM_SR_UIF;
   }
}
void EXTI0_1_IRQHandler(void) {
   // Your code to handle the EXTI interrupt for PA0 goes here
   // This is where you can perform specific actions when the rising edge occurs
        if (EXTI->RPR1 & EXTI_FPR1_FPIF0) // Check if the interrupt is from line 0
                {
                if(ledDirection == 0){
```

```
                        buttonCounter++;
                        if (buttonCounter > 10) {
                                buttonCounter = 9;
                                ledDirection = 1;
                        }
                }else if(ledDirection){
                        buttonCounter--;
                        if (buttonCounter < 1) {
                                buttonCounter = 2;
                                ledDirection = 0;
                        }
                }

                TIM2_Clock_Init();
                TIM2_Interrupt_Config();
                // Clear EXTI0 pending flag
                EXTI->RPR1 |= EXTI_FPR1_FPIF0;
                }
}
uint8_t readButton(void){

        return GPIOA->IDR & 1;
}
void EXTI_Init(void) {

   // Connect EXTI1 to GPIOA Pin 0
   EXTI->EXTICR[1] |= EXTI_EXTICR1_EXTI0_0;

   // Configure EXTI0 to rising edge trigger
   EXTI->RTSR1 |= EXTI_RTSR1_RT0;
   EXTI->FTSR1 &= ~EXTI_FTSR1_FT0;

   // Enable EXTI0 interrupt
   EXTI->IMR1 |= EXTI_IMR1_IM0;

   // Set the EXTI0 interrupt priority (adjust as needed)
   NVIC_SetPriority(EXTI0_1_IRQn, 1);
   NVIC_EnableIRQ(EXTI0_1_IRQn);
}

void GPIO_RCC_Init(void){
         /* Enable GPIOC clock */
        RCC->IOPENR |= (1U << 2);

        // Enable clock for GPIOA
        RCC->IOPENR |= RCC_IOPENR_GPIOAEN;
```

```
	// Enable clock for SYSCFG
	RCC->APBENR2 |= RCC_APBENR2_SYSCFGEN;

	// Enables GPIOA peripheral
	RCC->AHBENR |= 1;
}
void GPIOC_Init(void){
	//Configure GPIO for board on led
	GPIOC->MODER &= ~(3U << 2*6);
	GPIOC->MODER |= (1U << 2*6);
}
void GPIOA_Init(void){
	// Set GPIOA Pin 0 as input
	GPIOA->MODER &= ~GPIO_MODER_MODE0; // Clear mode bits for pin 0

	// Enable pull-down resistor for GPIOA Pin 0
	GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD0; // Clear pull-up/pull-down bits for pin 0
	GPIOA->PUPDR |= GPIO_PUPDR_PUPD0_1; // Set bit for pull-down

}
void toggle_led(void){
	GPIOC->ODR ^= (1U << 6);
}
void TIM2_Clock_Init(void){
	// Enable TIM2 clock
	RCC->APBENR1 |= RCC_APBENR1_TIM2EN;

	// Set TIM2 prescaler and period
	TIM2->PSC = TIMERPSC - 1;
	TIM2->CNT = 0;
	TIM2->ARR = (TIMERPERIYOD *buttonCounter) - 1;
}
void TIM2_Interrupt_Config(void){
	NVIC_SetPriority(TIM2_IRQn, 15);
	NVIC_EnableIRQ(TIM2_IRQn);
	//Update interrupt enable
	TIM2->DIER |= TIM_DIER_UIE;
}
void Start_TIM2(void){
	// Start TIM2
	TIM2->CR1 |= TIM_CR1_CEN;
}
void Stop_TIM2(void){
	// Stop TIM2
	TIM2->CR1 &= ~TIM_CR1_CEN;
```

```
}
```

# 3 PROBLEM3

In this problem, pushbutton control was provided on the seven segment display using external interrupts. When the button is pressed, the led on and the display starts to increase starting from 0, when it reaches 999, it stops and the led off. If the button is pressed again during the process, the display will reset and start over.

### 22.4.1 TIMx control register 1 (TIMx_CR1)(x = 2 to 4)

Address offset: 0x00

Reset value: 0x0000

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Res. | Res. | Res. | Res. | UIFRE MAP | Res. | CKD[1:0] | | ARPE | CMS[1:0] | | DIR | OPM | URS | UDIS | CEN |
| | | | | rw | | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 15:12 Reserved, must be kept at reset value.

Bit 11 **UIFREMAP**: UIF status bit remapping
0: No remapping. UIF status bit is not copied to TIMx_CNT register bit 31.
1: Remapping enabled. UIF status bit is copied to TIMx_CNT register bit 31.

Bit 10 Reserved, must be kept at reset value.

Bits 9:8 **CKD[1:0]**: Clock division
This bit-field indicates the division ratio between the timer clock (CK_INT) frequency and sampling clock used by the digital filters (ETR, TIx),
00: $t_{DTS} = t_{CK_INT}$
01: $t_{DTS} = 2 \times t_{CK_INT}$
10: $t_{DTS} = 4 \times t_{CK_INT}$
11: Reserved

Bit 7 **ARPE**: Auto-reload preload enable
0: TIMx_ARR register is not buffered
1: TIMx_ARR register is buffered

Bits 6:5 **CMS[1:0]**: Center-aligned mode selection
00: Edge-aligned mode. The counter counts up or down depending on the direction bit (DIR).
01: Center-aligned mode 1. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting down.
10: Center-aligned mode 2. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting up.
11: Center-aligned mode 3. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set both when the counter is counting up or down.

*Note: It is not allowed to switch from edge-aligned mode to center-aligned mode as long as the counter is enabled (CEN=1)*

Bit 4 **DIR**: Direction
0: Counter used as upcounter
1: Counter used as downcounter

*Note: This bit is read only when the timer is configured in Center-aligned mode or Encoder mode.*

Bit 3 **OPM**: One-pulse mode

0: Counter is not stopped at update event

1: Counter stops counting at the next update event (clearing the bit CEN)

Bit 2 **URS**: Update request source

This bit is set and cleared by software to select the UEV event sources.

0: Any of the following events generate an update interrupt or DMA request if enabled. These events can be:

- Counter overflow/underflow
- Setting the UG bit
- Update generation through the slave mode controller

1: Only counter overflow/underflow generates an update interrupt or DMA request if enabled.

Bit 1 **UDIS**: Update disable

This bit is set and cleared by software to enable/disable UEV event generation.

0: UEV enabled. The Update (UEV) event is generated by one of the following events:

- Counter overflow/underflow
- Setting the UG bit
- Update generation through the slave mode controller

Buffered registers are then loaded with their preload values.

1: UEV disabled. The Update event is not generated, shadow registers keep their value (ARR, PSC, CCRx). However the counter and the prescaler are reinitialized if the UG bit is set or if a hardware reset is received from the slave mode controller.

Bit 0 **CEN**: Counter enable

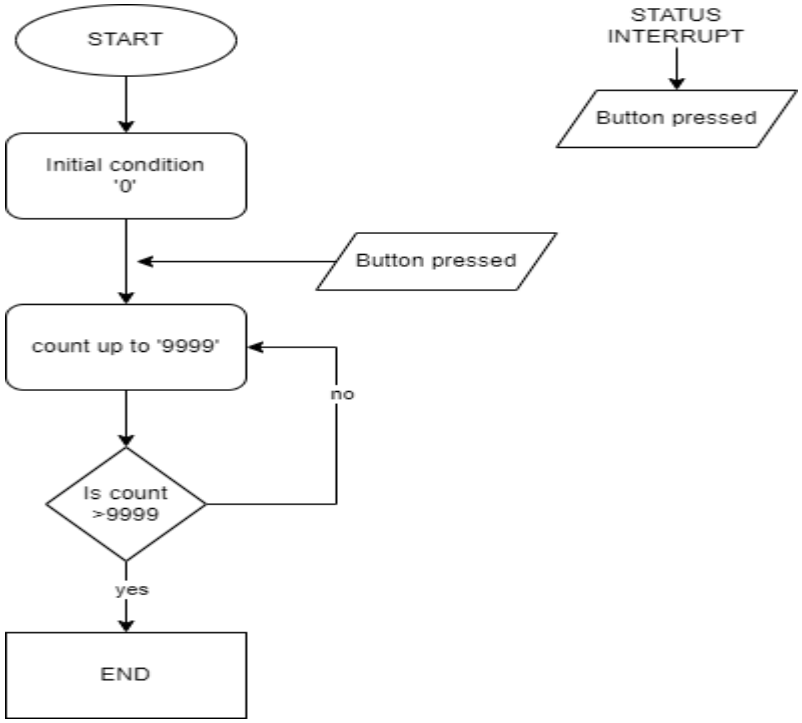
0: Counter disabled

1: Counter enabled

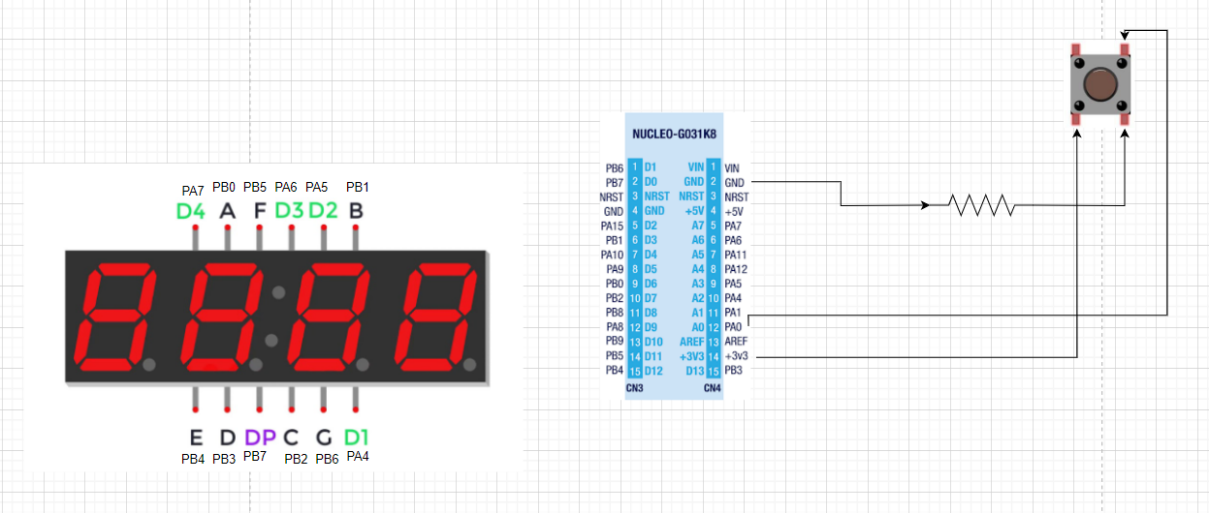
*Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.*

CEN is cleared automatically in one-pulse mode, when an update event occurs.

## 3.1 FLOWCHART



## 3.2 SCHEMATIC




## 3.3 CODE

```
#include "stm32g0xx.h"
#include "stm32g031xx.h"

// seven segment display defines
#define SEGMENT_A 1U << (1U - 1U)
#define SEGMENT_B 1U << (2U - 1U)
#define SEGMENT_C 1U << (3U - 1U)
#define SEGMENT_D 1U << (4U - 1U)
#define SEGMENT_E 1U << (5U - 1U)
#define SEGMENT_F 1U << (6U - 1U)
#define SEGMENT_G 1U << (7U - 1U)
#define SEGMENT_DP 1U << (8U - 1U)
#define SEGMENT_MASK ~(SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D |
SEGMENT_E | SEGMENT_F | SEGMENT_G | SEGMENT_DP)

#define DIGIT_1 1 << (4)
#define DIGIT_2 1 << (5)
#define DIGIT_3 1 << (6)
#define DIGIT_4 1 << (7)

#define DIGIT_MASK (DIGIT_1 | DIGIT_2 | DIGIT_3 | DIGIT_4)
// seven segment display numbers
uint8_t numbers[10] = {
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_E |
SEGMENT_F,                      // 0
        SEGMENT_B | SEGMENT_C,
                                                        // 1
```

```
        SEGMENT_A | SEGMENT_B | SEGMENT_D | SEGMENT_E | SEGMENT_G,
                                // 2
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_G,
                                // 3
        SEGMENT_B | SEGMENT_C | SEGMENT_F | SEGMENT_G,
                                    // 4
        SEGMENT_A | SEGMENT_C | SEGMENT_D | SEGMENT_F | SEGMENT_G,
                                // 5
        SEGMENT_A | SEGMENT_C | SEGMENT_D | SEGMENT_E | SEGMENT_F |
SEGMENT_G,                      // 6
        SEGMENT_A | SEGMENT_B | SEGMENT_C,
                                            // 7
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_E |
SEGMENT_F | SEGMENT_G, // 8
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_F |
SEGMENT_G,                      // 9
};
// defines
#define TIMERPSC 160
#define TIMERPERIYOD 25
// variables
uint64_t tick;
uint16_t numberCounter;
uint32_t counter;
uint8_t buttonIsPressed;

// functions prototypes
void GPIO_RCC_Init(void);
void GPIOA_Init(void);
void GPIOB_Init(void);
void GPIOC_Init(void);

void TIM2_Clock_Init(void);
void TIM2_Interrupt_Config(void);
void TIM2_IRQHandler(void);
void Start_TIM2(void);
void Stop_TIM2(void);

void toggle_led(void);
uint8_t readButton(void);

void EXTI_Init(void);
void EXTI0_1_IRQHandler(void);

void sevenSegmentShowNumber(uint16_t number);
void sevenSegmentShowDigit(uint8_t number, uint8_t digit);
```

```
void onboard_led_on(void);
void onboard_led_off(void);

int main(void)
{

	// init
	GPIO_RCC_Init();
	GPIOA_Init();
	GPIOB_Init();
	GPIOC_Init();
	//Show 0 at initial condition
	sevenSegmentShowDigit(0,3);
	TIM2_Clock_Init();
	TIM2_Interrupt_Config();
	EXTI_Init();

	Start_TIM2();

	while (1)
	{
		//sevenSegmentShowNumber(1234);
	}

	return 0;
}
void sevenSegmentShowNumber(uint16_t number)
{
	// define digits from the number and use GPIOB
	uint8_t digit1 = number / 1000;
	uint8_t digit2 = (number % 1000) / 100;
	uint8_t digit3 = (number % 100) / 10;
	uint8_t digit4 = number % 10;

	// select digit4 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;

	// set GPIOB to output for digit4 using numbers with 8 bit mask
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit4];
	GPIOA->ODR &= ~GPIO_ODR_OD7_Msk;

	// select digit3 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;

	// set GPIOB to output for digit3 using numbers
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit3];
```

```
	GPIOA->ODR &= ~GPIO_ODR_OD6_Msk;

	// select digit2 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;
	// set GPIOB to output for digit2 using numbers
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit2];
	GPIOA->ODR &= ~GPIO_ODR_OD5_Msk;

	// select digit1 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;
	// set GPIOB to output for digit1 using numbers
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit1];
	GPIOA->ODR &= ~GPIO_ODR_OD4_Msk;

}


void sevenSegmentShowDigit(uint8_t number, uint8_t digit)
{
	// select digit4 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;

	// set GPIOB to output for digit4 using numbers with 8 bit mask
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[number];
	GPIOA->ODR &= ~(1<< (digit + 4));

}

void TIM2_IRQHandler(void)
{
	if (TIM2->SR & TIM_SR_UIF)
	{
		if (buttonIsPressed){
			if (numberCounter<40000){
				numberCounter++;
				counter = numberCounter/4;
				onboard_led_off();
			}else{
				numberCounter = 0;
				counter = 0;
				buttonIsPressed = 0;
				onboard_led_on();
			}

			uint8_t digit1 = counter / 1000;
			uint8_t digit2 = (counter % 1000) / 100;
```

```
                        uint8_t digit3 = (counter % 100) / 10;
                        uint8_t digit4 = counter % 10;
                        if (numberCounter%4 == 0){
                                sevenSegmentShowDigit(digit4,3);
                        }else if(numberCounter%3 == 0){
                                sevenSegmentShowDigit(digit3,2);
                        }else if(numberCounter%2 == 0){
                                sevenSegmentShowDigit(digit2,1);
                        }else if(numberCounter%1 == 0){
                                sevenSegmentShowDigit(digit1,0);
                        }
                }


                //toggle_led();

                // Clear the interrupt flag
                TIM2->SR &= ~TIM_SR_UIF;
        }
}
void EXTI0_1_IRQHandler(void)
{
        // Your code to handle the EXTI interrupt for PA0 goes here
        // This is where you can perform specific actions when the rising edge occurs
        if (EXTI->RPR1 & EXTI_FPR1_FPIF0) // Check if the interrupt is from line 0
        {
                buttonIsPressed =1;


                // Clear EXTI0 pending flag
                EXTI->RPR1 |= EXTI_FPR1_FPIF0;
        }
}
uint8_t readButton(void)
{

        return GPIOA->IDR & 1;
}
void EXTI_Init(void)
{

        // Connect EXTI1 to GPIOA Pin 0
        EXTI->EXTICR[1] |= EXTI_EXTICR1_EXTI0_0;

        // Configure EXTI0 to rising edge trigger
        EXTI->RTSR1 |= EXTI_RTSR1_RT0;
```

```
	EXTI->FTSR1 &= ~EXTI_FTSR1_FT0;

	// Enable EXTI0 interrupt
	EXTI->IMR1 |= EXTI_IMR1_IM0;

	// Set the EXTI0 interrupt priority (adjust as needed)
	NVIC_SetPriority(EXTI0_1_IRQn, 1);
	NVIC_EnableIRQ(EXTI0_1_IRQn);
}

void GPIO_RCC_Init(void)
{
	/* Enable GPIOA ,B and C clock */
	RCC->IOPENR |= RCC_IOPENR_GPIOAEN | RCC_IOPENR_GPIOBEN |
RCC_IOPENR_GPIOCEN;

	// Enable clock for SYSCFG
	RCC->APBENR2 |= RCC_APBENR2_SYSCFGEN;

	// Enables GPIOA peripheral
	RCC->AHBENR |= 1;
}
void GPIOC_Init(void)
{
	// Configure GPIO for board on led
	GPIOC->MODER &= ~(3U << 2 * 6);
	GPIOC->MODER |= (1U << 2 * 6);
}
void GPIOB_Init(void)
{
	// clear GPIOB Pin 0,1,2,3,4,5,6,7,8
	GPIOB->MODER &= SEGMENT_MASK & SEGMENT_MASK << 8; // Clear mode bits
for pin 0
	// Set GPIOB Pin 0,1,2,3,4,5,6,7,8 as output
	GPIOB->MODER |= (1U << 2 * 0); // Set mode bits for pin 0
	GPIOB->MODER |= (1U << 2 * 1); // Set mode bits for pin 1
	GPIOB->MODER |= (1U << 2 * 2); // Set mode bits for pin 2
	GPIOB->MODER |= (1U << 2 * 3); // Set mode bits for pin 3
	GPIOB->MODER |= (1U << 2 * 4); // Set mode bits for pin 4
	GPIOB->MODER |= (1U << 2 * 5); // Set mode bits for pin 5
	GPIOB->MODER |= (1U << 2 * 6); // Set mode bits for pin 6
	GPIOB->MODER |= (1U << 2 * 7); // Set mode bits for pin 7
	GPIOB->MODER |= (1U << 2 * 8); // Set mode bits for pin 8
}
void GPIOA_Init(void)
{
```

```
	// Set GPIOA Pin 0 as input
	GPIOA->MODER &= ~GPIO_MODER_MODE0; // Clear mode bits for pin 0
	// clear GPIOA Pin 4,5,6,7
	GPIOA->MODER &= ~GPIO_MODER_MODE4; // Clear mode bits for pin 4
	GPIOA->MODER &= ~GPIO_MODER_MODE5; // Clear mode bits for pin 5
	GPIOA->MODER &= ~GPIO_MODER_MODE6; // Clear mode bits for pin 6
	GPIOA->MODER &= ~GPIO_MODER_MODE7; // Clear mode bits for pin 7
	//  Set GPIOA Pin 4,5,6,7 as output
	GPIOA->MODER |= (1U << 2 * 4); // Set mode bits for pin 4
	GPIOA->MODER |= (1U << 2 * 5); // Set mode bits for pin 5
	GPIOA->MODER |= (1U << 2 * 6); // Set mode bits for pin 6
	GPIOA->MODER |= (1U << 2 * 7); // Set mode bits for pin 7
	// Enable pull-down resistor for GPIOA Pin 0
	GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD0;   // Clear pull-up/pull-down bits for pin
0
	GPIOA->PUPDR |= GPIO_PUPDR_PUPD0_1; // Set bit for pull-down
}

void toggle_led(void)
{
	GPIOC->ODR ^= (1U << 6);
}

void onboard_led_on(void)
{
	GPIOC->ODR |= (1U << 6);
}
void onboard_led_off(void)
{
	GPIOC->ODR &= ~(1U << 6);
}
void TIM2_Clock_Init(void)
{
	// Enable TIM2 clock
	RCC->APBENR1 |= RCC_APBENR1_TIM2EN;

	// Set TIM2 prescaler and period
	TIM2->PSC = TIMERPSC - 1;
	TIM2->CNT = 0;
	TIM2->ARR = (TIMERPERIYOD)-1;
}
void TIM2_Interrupt_Config(void)
{
	NVIC_SetPriority(TIM2_IRQn, 15);
	NVIC_EnableIRQ(TIM2_IRQn);
	// Update interrupt enable
```

```
	TIM2->DIER |= TIM_DIER_UIE;
}

void Start_TIM2(void)
{
	// Start TIM2
	TIM2->CR1 |= TIM_CR1_CEN;
}
void Stop_TIM2(void)
{
	// Stop TIM2
	TIM2->CR1 &= ~TIM_CR1_CEN;
}
```

# 4 PROBLEM 4

For the independent watchdog timer, the time is set according to the following parameters and the blinking of the led is tested. If the set time is exceeded and the watchdog is not refreshed, the system will be reset.

### 28.4.1 IWDG key register (IWDG_KR)

Address offset: 0x00

Reset value: 0x0000 0000 (reset by Standby mode)

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
| | | | | | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| KEY[15:0] | | | | | | | | | | | | | | | |
| w | w | w | w | w | w | w | w | w | w | w | w | w | w | w | w |

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **KEY[15:0]:** Key value (write only, read 0x0000)

These bits must be written by software at regular intervals with the key value 0xAAAA, otherwise the watchdog generates a reset when the counter reaches 0.
Writing the key value 0x5555 to enable access to the IWDG_PR, IWDG_RLR and IWDG_WINR registers (see *Section 28.3.4: Register access protection*)
Writing the key value 0xCCCC starts the watchdog (except if the hardware watchdog option is selected)

### 28.4.2 IWDG prescaler register (IWDG_PR)

Address offset: 0x04

Reset value: 0x0000 0000

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
| | | | | | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | PR[2:0] | | |
| | | | | | | | | | | | | | rw | rw | rw |

Bits 31:3 Reserved, must be kept at reset value.

Bits 2:0 **PR[2:0]:** Prescaler divider

These bits are write access protected see *Section 28.3.4: Register access protection*. They are written by software to select the prescaler divider feeding the counter clock. PVU bit of the *IWDG status register (IWDG_SR)* must be reset in order to be able to change the prescaler divider.

000: divider /4
001: divider /8
010: divider /16
011: divider /32
100: divider /64
101: divider /128
110: divider /256
111: divider /256

*Note: Reading this register returns the prescaler value from the $V_{DD}$ voltage domain. This value may not be up to date/valid if a write operation to this register is ongoing. For this reason the value read from this register is valid only when the PVU bit in the IWDG status register (IWDG_SR) is reset.*

### 28.4.3 IWDG reload register (IWDG_RLR)

Address offset: 0x08

Reset value: 0x0000 0FFF (reset by Standby mode)

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
| | | | | | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| Res. | Res. | Res. | Res. | RL[11:0] | | | | | | | | | | | |
| | | | | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:12 Reserved, must be kept at reset value.

Bits 11:0 **RL[11:0]:** Watchdog counter reload value

These bits are write access protected see *Register access protection*. They are written by software to define the value to be loaded in the watchdog counter each time the value 0xAAAA is written in the *IWDG key register (IWDG_KR)*. The watchdog counter counts down from this value. The timeout period is a function of this value and the clock prescaler. Refer to the datasheet for the timeout information.
The RVU bit in the *IWDG status register (IWDG_SR)* must be reset to be able to change the reload value.

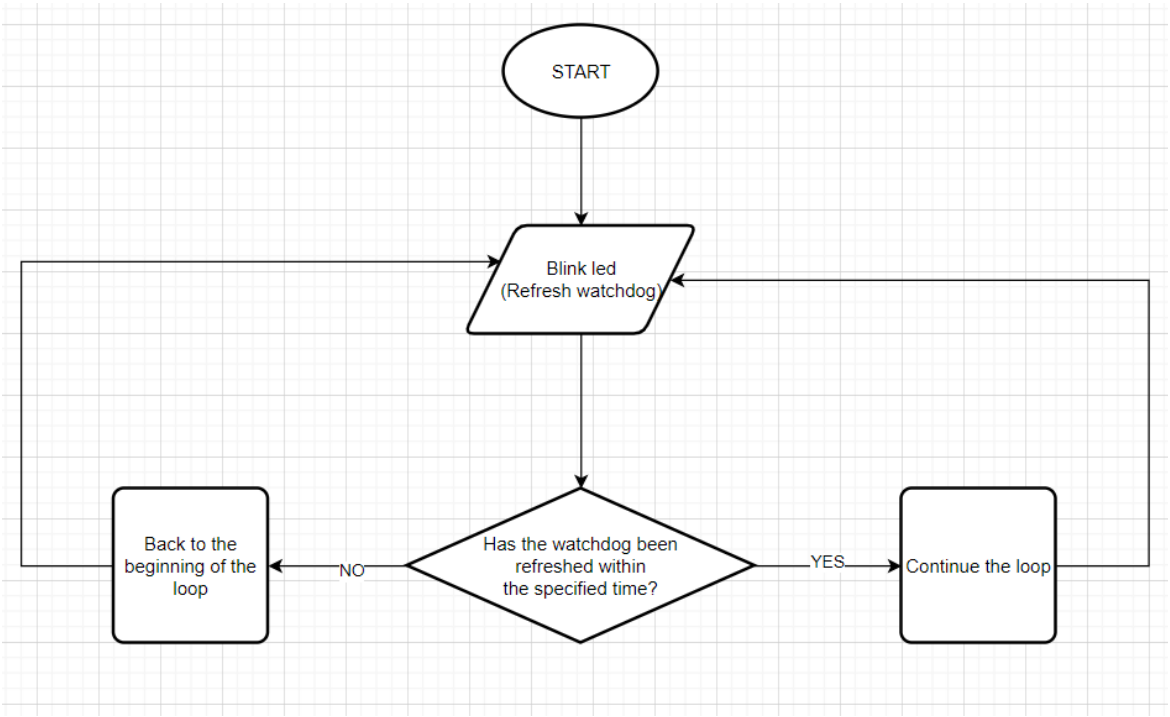
*Note: Reading this register returns the reload value from the $V_{DD}$ voltage domain. This value may not be up to date/valid if a write operation to this register is ongoing on it. For this reason the value read from this register is valid only when the RVU bit in the IWDG status register (IWDG_SR) is reset.*

- Setting IWDG time base:
  - IWDG time base prescaled from LSI clock (32 kHz)
    - 7 pre-dividers: 4 to 256 selectable by IWDG_PR register (and 12-bit watchdog counter reload value, RLR[11:0])
  - Setting the IWDG timeout by using the following formula:

$$t_{IWDG} = t_{LSI} \times 4 \times 2^{PR} \times (RL + 1)$$

where $t_{LSI}$ = 1/32000 = 31.25 µs, PR and RL are fields of IWDG registers

## 4.1 FLOWCHART




## 4.2 CODE

```
#include "stm32g0xx.h"

void IWDG_Init(void);
void IWDG_Refresh(void);
void SysTick_Handler(void);


void Init_Systick(uint32_t tick);

uint64_t getTick(void);
void delay_ms(uint64_t msvalue);

//variables
```

```
uint64_t tick;


#define LEDDELAYMS    300

void delay(volatile uint32_t);

int main(void) {
   SystemClock_Config();
   Init_Systick(16000);
   IWDG_Init();

   while (1) {
      // Your application code goes here
      //
            /* Enable GPIOC clock */
             RCC->IOPENR |= (1U << 2);

             /* Setup PC6 as output */
             GPIOC->MODER &= ~(3U << 2*6);
             GPIOC->MODER |= (1U << 2*6);

             /* Turn on LED */
             GPIOC->ODR |= (1U << 6);

             while(1) {
                   delay_ms(LEDDELAYMS);
                /* Toggle LED */
                GPIOC->ODR ^= (1U << 6);
                IWDG_Refresh();

             }

             return 0;


      // Reload the watchdog timer to prevent a reset

   }
}

void IWDG_Init(void) {
   // Enable write access to IWDG_PR and IWDG_RLR registers
   IWDG->KR = 0x5555;

   // Set prescaler and reload values
```

```
   IWDG->PR = 4;    // Prescaler: 4
   IWDG->RLR = 1000; // Reload value: 1000

   // Reload the watchdog counter
   IWDG->KR = 0xAAAA;

   // Enable the watchdog
   IWDG->KR = 0xCCCC;
}

void IWDG_Refresh(void) {
   // Reload the watchdog counter
   IWDG->KR = 0xAAAA;
}

void increase_tick(void){
        tick++;

}
void Init_Systick(uint32_t tick){
        SysTick->LOAD = tick; // Count down from 999 to 0
        SysTick->VAL  = 0;   // Clear current value
        SysTick->CTRL = 0x7; // Enable Systick, exception,and use processor clock
}
uint64_t getTick(void){

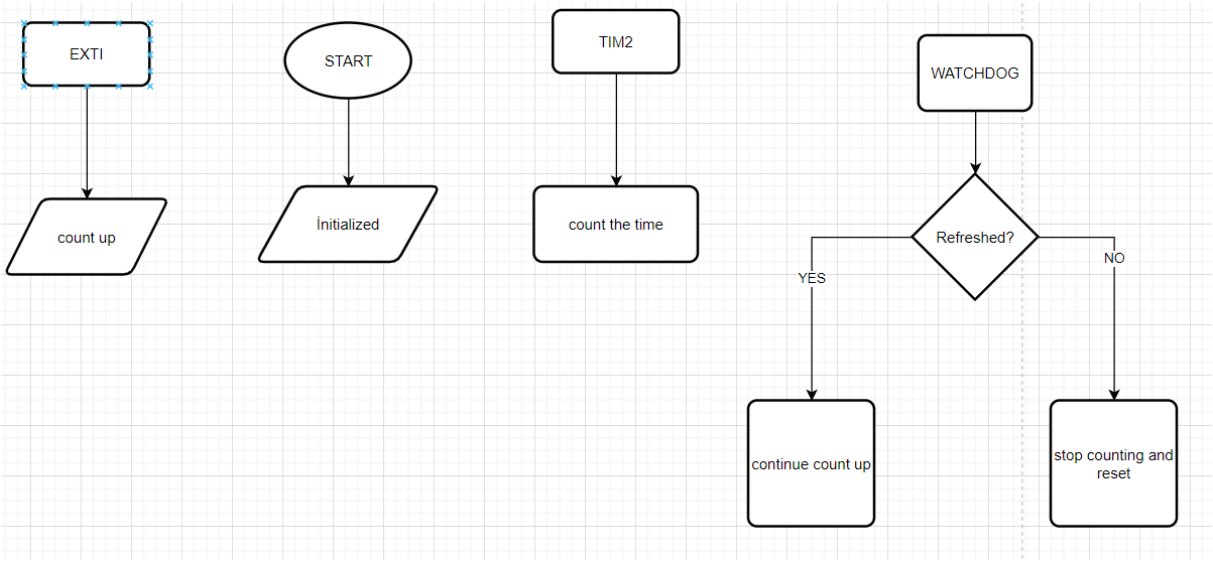
        return tick;
}
void delay_ms(uint64_t msvalue){
        uint64_t startTick =getTick() ;
        while ((getTick() - startTick) < msvalue)
         {
         }

}
/**
  * @brief This function handles System tick timer.
  */
void SysTick_Handler(void)
{
        increase_tick();
}
```

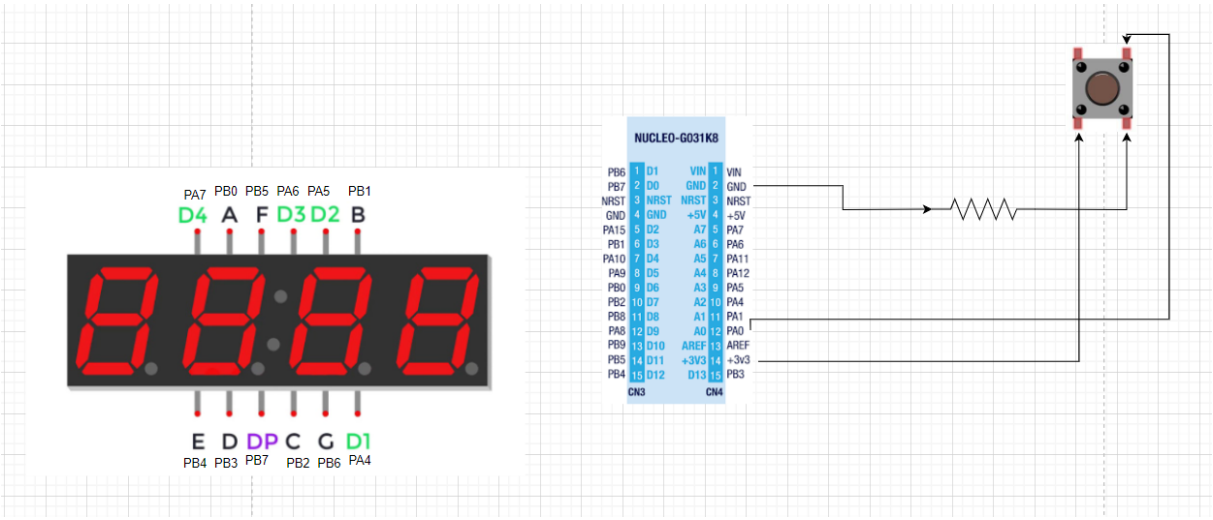
# 5 PROBLEM 5

The system was tested by adding watchdog timer while working as in problem 3. If the watchdog timer time is made 16 s and watchdog refresh is not done within this time, the screen is reset and starts counting from the beginning.

## 5.1 FLOWCHART




## 5.2 SCHEMATIC




## 5.3 CODE

```
#include "stm32g0xx.h"
#include "stm32g031xx.h"
```

```
// seven segment display defines
#define SEGMENT_A 1U << (1U - 1U)
#define SEGMENT_B 1U << (2U - 1U)
#define SEGMENT_C 1U << (3U - 1U)
#define SEGMENT_D 1U << (4U - 1U)
#define SEGMENT_E 1U << (5U - 1U)
#define SEGMENT_F 1U << (6U - 1U)
#define SEGMENT_G 1U << (7U - 1U)
#define SEGMENT_DP 1U << (8U - 1U)
#define SEGMENT_MASK ~(SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D |
SEGMENT_E | SEGMENT_F | SEGMENT_G | SEGMENT_DP)

#define DIGIT_1 1 << (4)
#define DIGIT_2 1 << (5)
#define DIGIT_3 1 << (6)
#define DIGIT_4 1 << (7)

#define DIGIT_MASK (DIGIT_1 | DIGIT_2 | DIGIT_3 | DIGIT_4)
// seven segment display numbers
uint8_t numbers[10] = {
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_E |
SEGMENT_F,                      // 0
        SEGMENT_B | SEGMENT_C,
                                                        // 1
        SEGMENT_A | SEGMENT_B | SEGMENT_D | SEGMENT_E | SEGMENT_G,
                                // 2
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_G,
                                // 3
        SEGMENT_B | SEGMENT_C | SEGMENT_F | SEGMENT_G,
                                        // 4
        SEGMENT_A | SEGMENT_C | SEGMENT_D | SEGMENT_F | SEGMENT_G,
                                // 5
        SEGMENT_A | SEGMENT_C | SEGMENT_D | SEGMENT_E | SEGMENT_F |
SEGMENT_G,                      // 6
        SEGMENT_A | SEGMENT_B | SEGMENT_C,
                                                // 7
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_E |
SEGMENT_F | SEGMENT_G, // 8
        SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_F |
SEGMENT_G,                      // 9
};
// defines
#define TIMERPSC 160
#define TIMERPERIYOD 25
#define WATCHDOG_PRESCALER 4
#define WATCHDOG_RELOAD 1000
```

```
// variables
uint64_t tick;
uint16_t numberCounter;
uint32_t counter;
uint8_t buttonIsPressed;

// functions prototypes
void GPIO_RCC_Init(void);
void GPIOA_Init(void);
void GPIOB_Init(void);
void GPIOC_Init(void);

void TIM2_Clock_Init(void);
void TIM2_Interrupt_Config(void);
void TIM2_IRQHandler(void);
void Start_TIM2(void);
void Stop_TIM2(void);

void toggle_led(void);
uint8_t readButton(void);

void EXTI_Init(void);
void EXTI0_1_IRQHandler(void);

void sevenSegmentShowNumber(uint16_t number);
void sevenSegmentShowDigit(uint8_t number, uint8_t digit);
void onboard_led_on(void);
void onboard_led_off(void);

void IWDG_Init(void);
void IWDG_Enable(void);

int main(void)
{

	// init
	GPIO_RCC_Init();
	GPIOA_Init();
	GPIOB_Init();
	GPIOC_Init();
	//Show 0 at initial condition
	sevenSegmentShowDigit(0,3);
	TIM2_Clock_Init();
	TIM2_Interrupt_Config();
	EXTI_Init();
```

```
	Start_TIM2();

	while (1)
	{
		//sevenSegmentShowNumber(1234);
	}

	return 0;
}
void sevenSegmentShowNumber(uint16_t number)
{
	// define digits from the number and use GPIOB
	uint8_t digit1 = number / 1000;
	uint8_t digit2 = (number % 1000) / 100;
	uint8_t digit3 = (number % 100) / 10;
	uint8_t digit4 = number % 10;

	// select digit4 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;

	// set GPIOB to output for digit4 using numbers with 8 bit mask
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit4];
	GPIOA->ODR &= ~GPIO_ODR_OD7_Msk;

	// select digit3 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;

	// set GPIOB to output for digit3 using numbers
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit3];
	GPIOA->ODR &= ~GPIO_ODR_OD6_Msk;

	// select digit2 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;
	// set GPIOB to output for digit2 using numbers
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit2];
	GPIOA->ODR &= ~GPIO_ODR_OD5_Msk;

	// select digit1 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;
	// set GPIOB to output for digit1 using numbers
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[digit1];
	GPIOA->ODR &= ~GPIO_ODR_OD4_Msk;

}
```

```
void sevenSegmentShowDigit(uint8_t number, uint8_t digit)
{
	// select digit4 using GPIOA with mask
	GPIOA->ODR |= DIGIT_MASK;

	// set GPIOB to output for digit4 using numbers with 8 bit mask
	GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[number];
	GPIOA->ODR &= ~(1<< (digit + 4));

}

void TIM2_IRQHandler(void)
{
	if (TIM2->SR & TIM_SR_UIF)
	{
		if (buttonIsPressed){
			if (numberCounter<40000){
				numberCounter++;
				counter = numberCounter/4;
				onboard_led_off();
			}else{
				numberCounter = 0;
				counter = 0;
				onboard_led_on();
			}

			uint8_t digit1 = counter / 1000;
			uint8_t digit2 = (counter % 1000) / 100;
			uint8_t digit3 = (counter % 100) / 10;
			uint8_t digit4 = counter % 10;
			if (numberCounter%4 == 0){
				sevenSegmentShowDigit(digit4,3);
			}else if(numberCounter%3 == 0){
				sevenSegmentShowDigit(digit3,2);
			}else if(numberCounter%2 == 0){
				sevenSegmentShowDigit(digit2,1);
			}else if(numberCounter%1 == 0){
				sevenSegmentShowDigit(digit1,0);
			}
		}


		//toggle_led();

		// Clear the interrupt flag
		TIM2->SR &= ~TIM_SR_UIF;
```

```
	}
}
void EXTI0_1_IRQHandler(void)
{
	// Your code to handle the EXTI interrupt for PA0 goes here
	// This is where you can perform specific actions when the rising edge occurs
	if (EXTI->RPR1 & EXTI_FPR1_FPIF0) // Check if the interrupt is from line 0
	{
		IWDG_Enable();
		buttonIsPressed = 1;

		// Clear EXTI0 pending flag
		EXTI->RPR1 |= EXTI_FPR1_FPIF0;
	}
}
uint8_t readButton(void)
{

	return GPIOA->IDR & 1;
}
void EXTI_Init(void)
{

	// Connect EXTI1 to GPIOA Pin 0
	EXTI->EXTICR[1] |= EXTI_EXTICR1_EXTI0_0;

	// Configure EXTI0 to rising edge trigger
	EXTI->RTSR1 |= EXTI_RTSR1_RT0;
	EXTI->FTSR1 &= ~EXTI_FTSR1_FT0;

	// Enable EXTI0 interrupt
	EXTI->IMR1 |= EXTI_IMR1_IM0;

	// Set the EXTI0 interrupt priority (adjust as needed)
	NVIC_SetPriority(EXTI0_1_IRQn, 1);
	NVIC_EnableIRQ(EXTI0_1_IRQn);
}

void GPIO_RCC_Init(void)
{
	/* Enable GPIOA ,B and C clock */
	RCC->IOPENR |= RCC_IOPENR_GPIOAEN | RCC_IOPENR_GPIOBEN |
RCC_IOPENR_GPIOCEN;

	// Enable clock for SYSCFG
	RCC->APBENR2 |= RCC_APBENR2_SYSCFGEN;
```

```
	// Enables GPIOA peripheral
	RCC->AHBENR |= 1;
}
void GPIOC_Init(void)
{
	// Configure GPIO for board on led
	GPIOC->MODER &= ~(3U << 2 * 6);
	GPIOC->MODER |= (1U << 2 * 6);
}
void GPIOB_Init(void)
{
	// clear GPIOB Pin 0,1,2,3,4,5,6,7,8
	GPIOB->MODER &= SEGMENT_MASK & SEGMENT_MASK << 8; // Clear mode bits
for pin 0
	// Set GPIOB Pin 0,1,2,3,4,5,6,7,8 as output
	GPIOB->MODER |= (1U << 2 * 0); // Set mode bits for pin 0
	GPIOB->MODER |= (1U << 2 * 1); // Set mode bits for pin 1
	GPIOB->MODER |= (1U << 2 * 2); // Set mode bits for pin 2
	GPIOB->MODER |= (1U << 2 * 3); // Set mode bits for pin 3
	GPIOB->MODER |= (1U << 2 * 4); // Set mode bits for pin 4
	GPIOB->MODER |= (1U << 2 * 5); // Set mode bits for pin 5
	GPIOB->MODER |= (1U << 2 * 6); // Set mode bits for pin 6
	GPIOB->MODER |= (1U << 2 * 7); // Set mode bits for pin 7
	GPIOB->MODER |= (1U << 2 * 8); // Set mode bits for pin 8
}
void GPIOA_Init(void)
{
	// Set GPIOA Pin 0 as input
	GPIOA->MODER &= ~GPIO_MODER_MODE0; // Clear mode bits for pin 0
	// clear GPIOA Pin 4,5,6,7
	GPIOA->MODER &= ~GPIO_MODER_MODE4; // Clear mode bits for pin 4
	GPIOA->MODER &= ~GPIO_MODER_MODE5; // Clear mode bits for pin 5
	GPIOA->MODER &= ~GPIO_MODER_MODE6; // Clear mode bits for pin 6
	GPIOA->MODER &= ~GPIO_MODER_MODE7; // Clear mode bits for pin 7
	//  Set GPIOA Pin 4,5,6,7 as output
	GPIOA->MODER |= (1U << 2 * 4); // Set mode bits for pin 4
	GPIOA->MODER |= (1U << 2 * 5); // Set mode bits for pin 5
	GPIOA->MODER |= (1U << 2 * 6); // Set mode bits for pin 6
	GPIOA->MODER |= (1U << 2 * 7); // Set mode bits for pin 7
	// Enable pull-down resistor for GPIOA Pin 0
	GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD0;   // Clear pull-up/pull-down bits for pin
0
	GPIOA->PUPDR |= GPIO_PUPDR_PUPD0_1; // Set bit for pull-down
}
```

```
void toggle_led(void)
{
	GPIOC->ODR ^= (1U << 6);
}

void onboard_led_on(void)
{
	GPIOC->ODR |= (1U << 6);
}
void onboard_led_off(void)
{
	GPIOC->ODR &= ~(1U << 6);
}
void TIM2_Clock_Init(void)
{
	// Enable TIM2 clock
	RCC->APBENR1 |= RCC_APBENR1_TIM2EN;

	// Set TIM2 prescaler and period
	TIM2->PSC = TIMERPSC - 1;
	TIM2->CNT = 0;
	TIM2->ARR = (TIMERPERIYOD)-1;
}
void TIM2_Interrupt_Config(void)
{
	NVIC_SetPriority(TIM2_IRQn, 15);
	NVIC_EnableIRQ(TIM2_IRQn);
	// Update interrupt enable
	TIM2->DIER |= TIM_DIER_UIE;
}

void Start_TIM2(void)
{
	// Start TIM2
	TIM2->CR1 |= TIM_CR1_CEN;
}
void Stop_TIM2(void)
{
	// Stop TIM2
	TIM2->CR1 &= ~TIM_CR1_CEN;
}
void IWDG_Init(void) {
   // Enable write access to IWDG_PR and IWDG_RLR registers
   IWDG->KR = 0x5555;

   // Set prescaler and reload values
```

```
    IWDG->PR = WATCHDOG_PRESCALER;    // Prescaler
    IWDG->RLR = WATCHDOG_RELOAD; // Reload value

    // Reload the watchdog counter
    IWDG->KR = 0xAAAA;

}
void IWDG_Enable(void){

    // Enable the watchdog
    IWDG->KR = 0xCCCC;
}
```

# 6 References


1. rm0444-stm32g0x1-advanced-armbased-32bit-mcus-stmicroelectronics
2. https://softwareparticles.com/learn-how-a-4-digit-7-segment-led-display-works-and-how-to-control-it-using-an-arduino/
3. https://documentation-service.arm.com/static/5e8e294afd977155116a6a5b?token=
4. https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf